

Sexism Detection

May 2, 2025

1 Sexism Classification with BLSTM and Transformer

In this notebook, we have implemented two BLSTM models and imported RoBERTa transformer. Then we used these to study a classification problem. The problem is defined as whether a given tweet is sexism.

1.1 Outlines

1. Packages
2. Corpus
3. Corpus Settings
4. Data Cleaning
5. Bidirectional LSTM Model
 1. Step 1 - Tokenization
 1. Download and Import GloVe
 2. Build Vocabulary - Word Tokenization
 3. Build Vocabulary - BPE
 2. Step 2 - Embeddings
 1. Words' ID and Tweets' Sequence IDs
 2. Checking OOV
 3. Embedding Matrix
 3. Step 3 - Paddings
 4. Step 4 - Model Definition
 5. Step 5 - Training
 6. Step 6 - Test Evaluation
6. Transformer Model
 1. Importing Transformer and Preprocessing
 2. Defining the model and Metrics
 3. Training
 4. Test Evaluation
7. Results Analysis
 1. BLSTM Curve Analysis
 2. Vocabulary and Tokenization Analysis
 3. Confusion Matrix Analysis
 4. Other Notes

Packages This sections will import and install all the necessary libraries for this project.

```
[ ]: # If Colab then uncomment this cell
# from google.colab import drive
# drive.mount("/content/drive")
```

```
[ ]: # Uncomment this cell if you don't have GloVe locally, O.W. later in the
# notebook you can change the path to yours
# Download GloVe embeddings
# Ref: https://paperswithcode.com/method/glove
# Following Glove Link is from its official Github account
# You can download them to your workspace by uncommenting the following lines

# Uncomment next two for the 840B tokens with 300D version of Glove
# !wget -p "<PATH_TO_SAVE>" https://huggingface.co/stanfordnlp/glove/resolve/
# main/glove.840B.300d.zip
# !unzip "<PATH_TO_SAVED>/glove.840B.300d.zip"

# Uncomment next two for the 6B tokens with 50D, 100D, 200D, 300D version of
# Glove
# !wget -p "<PATH_TO_SAVE>" https://huggingface.co/stanfordnlp/glove/resolve/
# main/glove.6B.zip
# !unzip "<PATH_TO_SAVED>/glove.6B.zip"
```

```
[ ]: # %%capture
# !pip install transformers datasets evaluate
```

```
[ ]: import json
import os
import random
import re
import warnings
from collections import Counter
from pathlib import Path

import evaluate
import matplotlib.pyplot as plt
import nltk
import numpy as np
import pandas as pd
import seaborn as sns
import tensorflow as tf
import torch
from datasets import Dataset as DS
from nltk.stem import WordNetLemmatizer
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, f1_score, precision_score,
                             recall_score)
from sklearn.utils import resample
```

```

from tensorflow.keras.layers import LSTM, Activation, BatchNormalization,
↳ Bidirectional, Dense, Dropout, Embedding
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing.sequence import pad_sequences
from torch.utils.data import Dataset
from transformers import (AutoModelForSequenceClassification, AutoTokenizer,
↳ DataCollatorWithPadding, Trainer,
                           TrainingArguments, set_seed)

warnings.filterwarnings("ignore", category=FutureWarning)

```

Corpus In this section we have downloaded the data from the [Github repository](#) and add it to our directory.

The three sets are slightly unbalanced, with a bias toward the Non-sexist class. In the following cells, I will discuss how I handled the balance and unbalance dataset.

```

[ ]: # Updated the following paths base on your own environment
path_to_train_json = "<PATH_TO>/training.json"
path_to_val_json = "<PATH_TO>/validation.json"
path_to_test_json = "<PATH_TO>/test.json"

def load_json_to_df(file_path):
    with open(file_path, "r") as f:
        data = json.load(f)
    return pd.DataFrame.from_dict(data, orient="index")

# Load the JSON files into pandas DataFrames
train_df = load_json_to_df(path_to_train_json)
val_df = load_json_to_df(path_to_val_json)
test_df = load_json_to_df(path_to_test_json)

print("Train df shape:", train_df.shape)
print("Validation df shape:", val_df.shape)
print("Test df shape:", test_df.shape)

train_df.head(3)

```

Train df shape: (6920, 11)

Validation df shape: (726, 11)

Test df shape: (312, 11)

```

[ ]:      id_EXIST lang      tweet \
100001  100001  es  @TheChiflis Ignora al otro, es un capullo.El p...
100002  100002  es  @ultimonomada_ Si comicsgate se parece en algo...

```

```

100003    100003    es    @Steven2897 Lee sobre Gamergate, y como eso ha...

        number_annotators                                annotators \
100001                6    [Annotator_1, Annotator_2, Annotator_3, Annota...
100002                6    [Annotator_7, Annotator_8, Annotator_9, Annota...
100003                6    [Annotator_7, Annotator_8, Annotator_9, Annota...

        gender_annotators                                age_annotators \
100001    [F, F, F, M, M, M]    [18-22, 23-45, 46+, 46+, 23-45, 18-22]
100002    [F, F, F, M, M, M]    [18-22, 23-45, 46+, 46+, 23-45, 18-22]
100003    [F, F, F, M, M, M]    [18-22, 23-45, 46+, 46+, 23-45, 18-22]

        labels_task1 \
100001    [YES, YES, NO, YES, YES, YES]
100002        [NO, NO, NO, NO, YES, NO]
100003        [NO, NO, NO, NO, NO, NO]

        labels_task2 \
100001    [REPORTED, JUDGEMENTAL, -, REPORTED, JUDGEMENT...
100002                [-, -, -, -, DIRECT, -]
100003                [-, -, -, -, -, -]

        labels_task3    split
100001    [[OBJECTIFICATION], [OBJECTIFICATION, SEXUAL-V... TRAIN_ES
100002        [[-], [-], [-], [-], [OBJECTIFICATION], [-]] TRAIN_ES
100003        [[-], [-], [-], [-], [-], [-]] TRAIN_ES

```

Corpus Settings In this step, we will modify the dataset with defined rules settings.

```

[ ]: def majority_voting(labels):
    count = Counter(labels)
    if count["YES"] > count["NO"]:
        return "YES"
    elif count["NO"] > count["YES"]:
        return "NO"
    else:
        return None

def initial_process(df):
    # Generate hard labels using majority voting
    df["hard_label_task1"] = df["labels_task1"].apply(majority_voting)

    # Remove rows without a clear majority
    df = df.dropna(subset=["hard_label_task1"])

    # Filter for English tweets

```

```

df = df[df["lang"] == "en"]

# Keep only relevant columns
df = df[["id_EXIST", "lang", "tweet", "hard_label_task1"]]

# Encode the hard labels
df["hard_label_task1"] = df["hard_label_task1"].map({"YES": 1, "NO": 0})

# Display the resulting DataFrame
return df

train_df = initial_process(train_df)
val_df = initial_process(val_df)
test_df = initial_process(test_df)

print("Train df shape:", train_df.shape)
print("Validation df shape:", val_df.shape)
print("Test df shape:", test_df.shape)
print("\n=== Number of datapoints per dataset&class ===")
print("Training Set:")
print(train_df["hard_label_task1"].value_counts())
print("\nValidation Set:")
print(val_df["hard_label_task1"].value_counts())
print("\nTest Set:")
print(test_df["hard_label_task1"].value_counts())

```

```

Train df shape: (2870, 4)
Validation df shape: (158, 4)
Test df shape: (286, 4)

```

```

=== Number of datapoints per dataset&class ===

```

```

Training Set:

```

```

hard_label_task1

```

```

0    1733

```

```

1    1137

```

```

Name: count, dtype: int64

```

```

Validation Set:

```

```

hard_label_task1

```

```

0     90

```

```

1     68

```

```

Name: count, dtype: int64

```

```

Test Set:

```

```

hard_label_task1

```

```

0    160

```

```

1    126

```

Name: count, dtype: int64

Results above shows the the number of datapoints that we have for the `none sexist` class, class 0, is slightly more than the other one.

We didn't get rid of this unbalanced dataset in this project. In fact, using this unbalance dataset, we have created another training dataset in which the class with higher frequency is downsampled to the other class and we have a balance number of both. Hence, from now on we have two datasets:

- `train_df`: The main dataset with its unbalancy.
- `train_df_balanced`: The new dataset in which we have a same amount of classes.

In this project we will use both of these datasets to train the defined models and we will discuss and analyse them together.

Next cell, creates this second balanced dataset with downsampling. Moreover, the number of data is few, and downsampling narrows it even more. Hence, a better approach would be to upsample the data of the class with fewer frequency, which can be done in future experiments.

```
[ ]: # Separate majority and minority classes
df_majority = train_df[train_df["hard_label_task1"] == 0]
df_minority = train_df[train_df["hard_label_task1"] == 1]

# Downsample majority class
df_majority_downsampled = resample(
    df_majority,
    replace=False, # sample without replacement
    n_samples=len(df_minority), # to match minority class
    random_state=123,
) # reproducible results

# Combine minority class with downsampled majority class
train_df_balanced = pd.concat([df_majority_downsampled, df_minority])

# Display new class counts
print("=== Number of datapoints per dataset&class after downsampling ===")
print(train_df_balanced["hard_label_task1"].value_counts())
train_df_balanced.info()
```

```
=== Number of datapoints per dataset&class after downsampling ===
```

```
hard_label_task1
```

```
0    1137
```

```
1    1137
```

```
Name: count, dtype: int64
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 2274 entries, 200685 to 203260
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	id_EXIST	2274 non-null	object
1	lang	2274 non-null	object

```

2   tweet                2274 non-null   object
3   hard_label_task1    2274 non-null   int64
dtypes: int64(1), object(3)
memory usage: 88.8+ KB

```

```
[ ]: train_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 2870 entries, 200002 to 203260
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id_EXIST              2870 non-null   object
1   lang                  2870 non-null   object
2   tweet                 2870 non-null   object
3   hard_label_task1     2870 non-null   int64
dtypes: int64(1), object(3)
memory usage: 176.6+ KB

```

Data Cleaning In the context of tweets, we have noisy and informal data that often includes unnecessary elements like emojis, hashtags, mentions, and URLs. These elements may interfere with the text analysis. Although these may be important in other tasks, here we will drop them from the data.

We will do a small data cleaning to omit the parts of the tweet that do not contain any information. This is done by the following `clean_tweet` function which takes a tweet in and returns it cleaner than ever ;).

```
[ ]: def clean_tweet(tweet):
    # Remove(substitute with "") emojis and non Ascii
    tweet = re.sub(r"[^\x00-\x7F]+", "", tweet)

    # Remove hashtags or complex usage of hashtags
    tweet = re.sub(r"#+\w+|#+", "", tweet)

    # Remove mentions or more complex uses of @
    tweet = re.sub(r"@+\w+|@+", "", tweet)

    # Remove URLs
    tweet = re.sub(r"http\S+|www\S+|https\S+", "", tweet)

    # Remove special characters; Look for something, remove something else
    tweet = re.sub(r"[^\w\s]", "", tweet)

    # Remove specific quote characters; less probable to occur
    tweet = re.sub(r"[\u2018\u2019\u201C\u201D]", "", tweet)

    return tweet

```

```

# Apply the cleaning function to the 'tweet' column
train_df["cleaned_tweet"] = train_df["tweet"].apply(clean_tweet)
train_df_balanced["cleaned_tweet"] = train_df_balanced["tweet"].
    ↪ apply(clean_tweet)
val_df["cleaned_tweet"] = val_df["tweet"].apply(clean_tweet)
test_df["cleaned_tweet"] = test_df["tweet"].apply(clean_tweet)

```

In the next cell, we have used the `nlTK` tool to lemmatize the input text to their base form. This process, as well as the cleaning process we saw in the previous cell, is being applied to the both of the training sets along with validation and test sets.

```

[ ]: # Lemmatization
nlTK.download("wordnet")
nlTK.download("omw-1.4")

lemmatizer = WordNetLemmatizer()

def lemmatize_text(text):
    """
    inputs:
        - text: the input tweet
    outputs:
        - lemmatized_words: the lemmatized tweet
    """
    words = text.split()
    lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
    return " ".join(lemmatized_words)

# Apply the cleaning function to the 'tweet' column
train_df["lemmatized_tweet"] = train_df["cleaned_tweet"].apply(lemmatize_text)
train_df_balanced["lemmatized_tweet"] = train_df_balanced["cleaned_tweet"].
    ↪ apply(lemmatize_text)
val_df["lemmatized_tweet"] = val_df["cleaned_tweet"].apply(lemmatize_text)
test_df["lemmatized_tweet"] = test_df["cleaned_tweet"].apply(lemmatize_text)
train_df.shape, train_df_balanced.shape

```

```

[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...

```

```

[ ]: ((2870, 6), (2274, 6))

```

Bidirectional LSTM Model In this section we will go through step to step preparation of our architecure and we will implements necessary parts that have to be done before doing training.

Step 1 - Tokenization We have preprocessed the entire data we had. Now it's time tokenize the data and create our vocabulary. We can do this part with two approach here: - One approach is to

just simply consider all the word type (words that are unique) and are processed and lemmatized as a token and add them to vocabulary list. - Second approach is to choose another method for tokenization. Like BPE. The second approach is a good approach which can be easily implemented with the provided functions by the `SentencePiece`. BPE is a well known algorithm which, by considering a smaller parts of a word as a token, decrease the probability of encountering the OOV words.

However, for the simplicity, we have chosen the first approach. We added all the word types appeared in the training set to the vocabulary. Moreover, since we are using GloVe, we also added the words of the GloVe to the vocabulary as well which helps us encountering less OOV and use the existing knowledge.

Our vocabulary for this project consists of the union of all the word types in the training set and in the GloVe embeddings dataset.

Step 1.1 - Download and import GloVe Importing the GloVe to the project.

```
[ ]: # Load GloVe embeddings
glove_embeddings = {}
embedding_dim = 100 # ensure the vector has the correct dim before converting
                    ↪and storing

# Update the following path base on your own environment
path_to_glove = (
    "<PATH_TO_FILE>/glove.6B.100d.txt"
)
with open(path_to_glove, "r", encoding="utf-8") as f:
    cnt = 0
    for line in f:
        values = line.split()
        word = values[0]
        if len(values[1:]) == embedding_dim:
            try:
                vector = np.asarray(values[1:], "float32")
            except ValueError:
                continue
            glove_embeddings[word] = vector
```

Step 1.2 - Building Vocabulary - Word Tokenization In the next cell, we will create the vocabulary list as explained before. Having said that, since we have two training datasets, we need to also have two different vocabularies as the number of words or even word types may vary.

```
[ ]: def create_vocabulary_WT(df, embedding_dictionary):
    """Word Tokenization
    inputs:
        - df: It is the dataframe of training set that contains the tweets
        - embedding_dictionary: the dictionary mapping the words to embeddings
    """
    train_tokens = []
```

```

for tweet in df["lemmatized_tweet"]:
    train_tokens.extend(tweet.split())
vocabulary = set(train_tokens) # Removing the multiple occurrences

# Adding Glove words to the vocabulary
vocabulary.update(glove_embeddings.keys())
return vocabulary

```

Creating vocabulary list from the training test helps us to not encounter [UNK] words while training the model. Adding the Glove tokens to the vocabulary list will also helps us to use the existing knowledge for training. Since, [UNK] words are expensive and affects the performance, using the existing knowledge helps the model to encounter to [UNK] fewer time and as the result understand the text and data better.

```

[ ]: # Create vocabulary from training set
vocabulary = create_vocabulary_WT(train_df, glove_embeddings)
vocab_count = len(vocabulary)
print("Length of vocabulary of union GloVe and Training data: ", vocab_count)

vocabulary_balanced = create_vocabulary_WT(train_df_balanced, glove_embeddings)
vocab_balanced_count = len(vocabulary_balanced)
print(
    "Length of vocabulary_balanced of union GloVe and Training data: ",
    vocab_balanced_count,
)

```

```

Length of vocabulary of union GloVe and Training data: 405482
Length of vocabulary_balanced of union GloVe and Training data: 404439

```

Step 2 - Embeddings Architectures work with numerical values. Hence, in this second step, our task is to convert each token to their numerical representation, which in practice are called embeddings. - For the embeddings, we will use the [GloVe Embeddings](#). - This process can be handle easily by only going through each token in each tweet in each dataset, and then find that token in the Glove, and choose that embeddings defined in the GloVe for that token. - In this step, we also create an embedding matrix, which simply is a matrix of dimension (length of vocabulary) * (Length of embedding).

Step 2.1 - Words' ID and Tweets' Sequence IDs The next cell will implement `word_signature` function which is responsible for defining an ID for each token in the vocabulary. Having said that, we also do have [PAD] (makes sure sequences of input has same shape) and [UNK] tokens, for special situation. Hence, we will also add these two tokens and their corresponding IDs to the vocabulary as well.

```

[ ]: def word_signature(vocabulary, PAD_TOKEN, UNK_TOKEN):
    word_to_index = {}
    index_to_word = {}

    # Convert the vocabulary set to a list for stable ordering
    vocab_list = sorted(list(vocabulary))

```

```

for idx, word in enumerate(vocab_list, start=2):
    word_to_index[word] = idx
    index_to_word[idx] = word

# adding [pad] token with id 0
word_to_index[PAD_TOKEN] = 0
index_to_word[0] = PAD_TOKEN

# adding [unk] token with id 1
word_to_index[UNK_TOKEN] = 1
index_to_word[1] = UNK_TOKEN

return word_to_index, index_to_word

```

```

[ ]: UNK_TOKEN = "<UNK>"
PAD_TOKEN = "<PAD>"
UNK_EMBED = np.random.rand(embedding_dim)

word_to_index, index_to_word = word_signature(vocabulary, PAD_TOKEN, UNK_TOKEN)
word_to_index_balanced, index_to_word_balanced =  word_signature(vocabulary_balanced, PAD_TOKEN, UNK_TOKEN)

# Updating size of vocabularies due to adding two special tokens
vocab_count = len(word_to_index)
vocab_balanced_count = len(word_to_index_balanced)

print("Final vocabulary size (with PAD, UNK):", vocab_count)
print("Final vocabulary balanced size (with PAD, UNK):", vocab_balanced_count)

```

Final vocabulary size (with PAD, UNK): 405484

Final vocabulary balanced size (with PAD, UNK): 404441

Now that we have defined corresponding IDs for each token, in the following we will define the sequence of IDs for each tweet. In other words, we have implemented the **text to sequence** function which in input takes a text(a tweet) along with the dictionary of mapping word to index. Then, for each word in the tweet, it replaces it with its corresponding ID and returns the corresponding sequences of IDs for the given tweet. - Keep in mind, since we have two training data, this has to be done twice for each dataset. Because the generated vocabulary list has different words and different IDs for words as a consequence. - Another important thing that needs to be mentioned is the unknown token. When we are applying this function to the validation and test datasets, we may encounter words in these two datasets that are not in the vocabulary. Hence, for these words, we are assigning the ID of a special token, the unknown token we discussed earlier.

```

[ ]: def text_to_sequence(text, word_to_index, unk_token=UNK_TOKEN):
    tokens = text.split() # Since we simply considered each word as a token
    seq = []
    for token in tokens:

```

```

        if token in word_to_index.keys():
            seq.append(word_to_index[token]) # if token is known, add its ID
        else: # then it is unknown, add UNK_TOKEN ID
            seq.append(word_to_index[unk_token])
    return seq

# ----- Unbalanced Data
train_sequences = []
for tweet in train_df["lemmatized_tweet"]:
    train_sequences.append(text_to_sequence(tweet, word_to_index))
val_sequences = []
for tweet in val_df["lemmatized_tweet"]:
    val_sequences.append(text_to_sequence(tweet, word_to_index))
test_sequences = []
for tweet in test_df["lemmatized_tweet"]:
    test_sequences.append(text_to_sequence(tweet, word_to_index))

# ----- Balanced Data
train_balanced_sequences = []
for tweet in train_df_balanced["lemmatized_tweet"]:
    train_balanced_sequences.append(text_to_sequence(tweet, word_to_index_balanced))
val_balanced_sequences = []
for tweet in val_df["lemmatized_tweet"]:
    val_balanced_sequences.append(text_to_sequence(tweet, word_to_index_balanced))
test_balanced_sequences = []
for tweet in test_df["lemmatized_tweet"]:
    test_balanced_sequences.append(text_to_sequence(tweet, word_to_index_balanced))

```

Step 2.2 - Checking OOV Time to check out the status of Words that are not in the vocabulary. Since we have created our vocabulary based on the words exists in the unipn of the training text and GloVe embeddings, we cannot expect existence of OOV in the training set. However, same doesn't apply for the test set and validation set which we have checked in the following:

```

[ ]: def check_OOV(sequences, word_to_index):
        oov_count = 0
        for sequence in sequences:
            for token_id in sequence:
                if index_to_word[token_id] == UNK_TOKEN:
                    oov_count += 1
        return oov_count

[ ]: print("OOV in unbalanced training set:", check_OOV(train_sequences, index_to_word))

```

```
print("OOV in unbalanced validation set:", check_OOV(val_sequences,
↪index_to_word))
print("OOV in unbalanced test set:", check_OOV(test_sequences, index_to_word))
```

```
OOV in unbalanced training set: 0
OOV in unbalanced validation set: 295
OOV in unbalanced test set: 456
```

```
[ ]: print("OOV in balanced training set:", check_OOV(train_balanced_sequences,
↪index_to_word))
print("OOV in balanced validation set:", check_OOV(val_balanced_sequences,
↪index_to_word))
print("OOV in balanced test set:", check_OOV(test_balanced_sequences,
↪index_to_word))
```

```
OOV in balanced training set: 0
OOV in balanced validation set: 315
OOV in balanced test set: 473
```

These numbers are the total number of OOV tokens. For the validation set, the model will encounter 315 words that are unknown for it.

Step 2.3 - Embedding Matrix An embedding layer needs a embedding matrix. In the next cell, we are defining the embedding matrix, which simply is a matrix of tokens by their corresponding embedding.

In the following cell, since the embedding matrix is initially being created based on vocabulary, we don't need to take care of the [UNK] as they don't happen. Moreover, we are using a fix embedding for the unknown words. Also, words that came from training set, may not be in GloVe, which means that we don't have any embeddings defined for them in the Glove. For these words we have choosen a random numbers from a normal distribution.

A better and more professional way for deciding the embedding of the words that are not in the GloVe could be to similarity techniques, to take the neighbour close words to that words hoping that maybe it captures some meaning and characteristics from them.

```
[ ]: def embedding_matrix_generator(embedding_dim, word_to_index, PAD_TOKEN,
↪UNK_TOKEN, UNK_EMBED):
    # Matrix Initialization
    embedding_matrix = np.zeros((len(word_to_index), embedding_dim))

    for word, idx in word_to_index.items():
        if word == PAD_TOKEN:
            continue # Leave it zero
        elif word == UNK_TOKEN:
            embedding_matrix[idx] = UNK_EMBED # is defined earlier; fixed;
↪randomly created
        elif word in glove_embeddings.keys():
            embedding_matrix[idx] = glove_embeddings[word]
        else:
```

```

        # Word not found in GloVe; we define a representation
        embedding_matrix[idx] = np.random.normal(scale=0.01,
        ↪size=(embedding_dim,))

    return embedding_matrix

```

```

[ ]: embedding_matrix = embedding_matrix_generator(embedding_dim, word_to_index,
        ↪PAD_TOKEN, UNK_TOKEN, UNK_EMBED)

embedding_matrix_balanced = embedding_matrix_generator(
    embedding_dim, word_to_index_balanced, PAD_TOKEN, UNK_TOKEN, UNK_EMBED
)

```

Step 3 - Padding Each input is a sequence of text that may have different number of tokens and different length. The model cannot handle these kind of variations and needs a fixed input in terms of length. Hence, we need to make sure that the sequences all have the same length. Generally this process is called padding. It involves, increasing or shrinking the sequences of tweets.

This can be achieved easily by defining a maximum length sequence parameter. Based on this variable, we convert the input sequences to have a sequence of length **maximum length**.

We will define maximum length parameter to be equal to the 95% of sequences length.

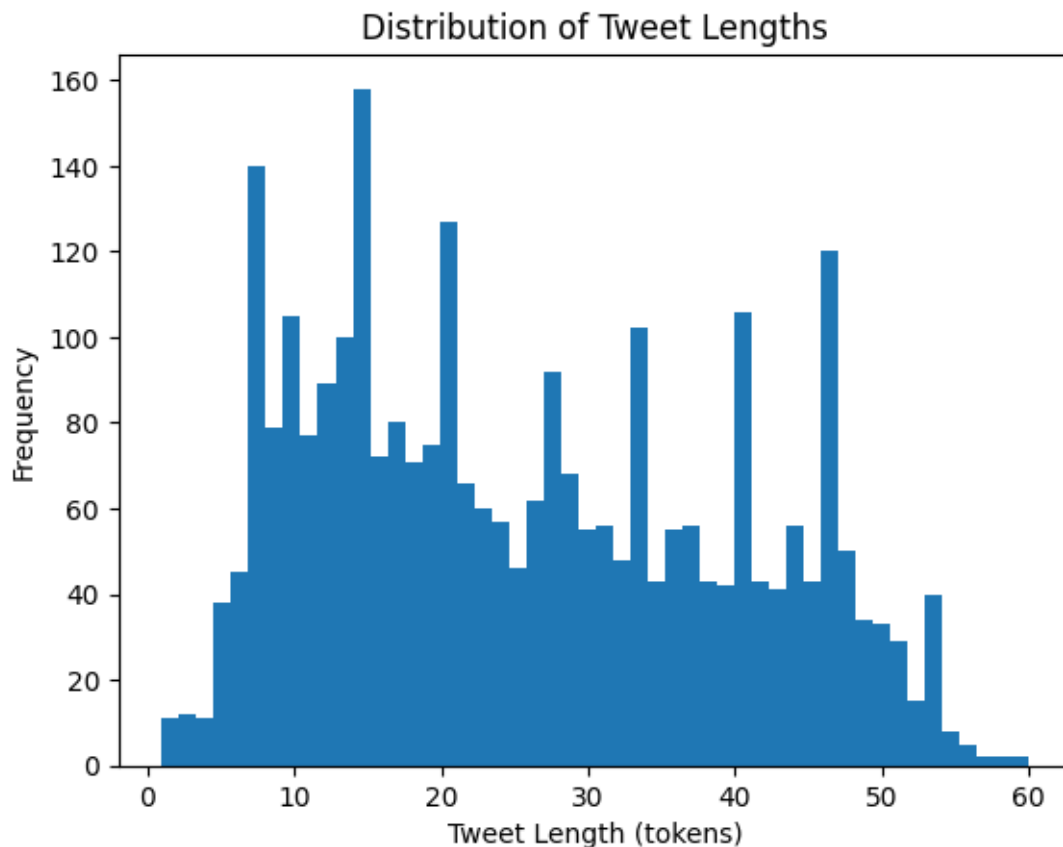
```

[ ]: # Testing on unbalanced dataset
lengths = [len(seq) for seq in train_sequences]
max_length = int(np.percentile(lengths, 95))
print("The length of the 95 of the sequences is", max_length)

plt.hist(lengths, bins=50)
plt.xlabel("Tweet Length (tokens)")
plt.ylabel("Frequency")
plt.title("Distribution of Tweet Lengths")
plt.show()

```

The length of the 95 of the sequences is 49



The padding can simply be done with the following code snippet. This code also, truncates the sequences larger than defined max length to have the same length.

```
[ ]: train_padded = pad_sequences(train_sequences, maxlen=max_length, padding="post")
val_padded = pad_sequences(val_sequences, maxlen=max_length, padding="post")
test_padded = pad_sequences(test_sequences, maxlen=max_length, padding="post")

train_balanced_padded = pad_sequences(train_balanced_sequences,
    ↪maxlen=max_length, padding="post")
val_balanced_padded = pad_sequences(val_balanced_sequences, maxlen=max_length,
    ↪padding="post")
test_balanced_padded = pad_sequences(test_balanced_sequences,
    ↪maxlen=max_length, padding="post")

print(train_padded[1:4])
print()
print()
print(train_balanced_padded[4:7])
```

```
[[198405 197657 17787 269440 13925 44748 132143 62132 362478 274716
```

```

373654 313368 92437 259421 400043 359724 245505 230410 76337 243923
58939 65522 362533 237326 76337 187187 238514 193146 259125 283053
45583 193012 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0]
[ 43123 366222 47188 118536 44748 179253 291603 272955 365186 366222
167808 344485 362533 46449 109002 179812 388448 366222 284253 243923
394131 193146 44039 44962 227224 61407 277518 394825 44800 353615
272955 198405 193146 311848 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0]
[ 46362 275128 82705 133848 76111 46384 92085 191304 399945 63303
47188 82705 399945 133848 393568 61939 55695 58939 390611 193146
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0]]

[[ 45949 46077 45095 43648 155028 310241 258205 221776 258205 21803
288363 273787 44582 6768 45340 58145 11440 121959 43719 45340
44058 45440 45833 44489 43826 274969 45451 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0]
[189234 394559 323836 361542 384102 84814 196769 127378 155028 243017
91274 197517 122951 253648 59349 365225 148812 361385 361542 393679
386194 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0]
[158455 59348 226330 121886 105121 393795 391268 62443 340330 324255
272027 368064 284977 356289 67789 361630 312687 391371 190425 341487
196769 368306 365225 74562 391268 362093 62443 58145 197517 130446
188097 398905 274969 60628 139488 189234 131305 327487 186313 197517
235345 46404 127351 150382 197207 167453 365225 74562 125670]]

```

Output proves that for sequences of tweets with less than max defined length, we are padding those sequences by zero to have same shape.

Step 4 - Model definition In the following we will define the sexism classifier and model architecture. We will have two different modelings. One is called **Base model U** which has one layer of Bi-LSTM with 64 units and the other model has an additional layer of Bi-LSTM with 32 units. However, since we kept both the balanced and unbalanced data, we will have the four following models: - **Base model U** - Utilized for the unbalanced data - **Model UA** - contains an additional layer compared to its successor - **Base model B** - Utilized for the balanced data - **Model BA** - contains an additional layer compared to its successor

```

[ ]: # Model U - Model for Unbalanced data ; has different embeddings
embedding_layer = Embedding(
    input_dim=vocab_count,
    output_dim=embedding_dim,
    weights=[embedding_matrix],

```



```

        trainable=False,
    )

# Model B - Model for Balanced data ; has different embeddings
embedding_balanced_layer = Embedding(
    input_dim=vocab_balanced_count,
    output_dim=embedding_dim,
    weights=[embedding_matrix_balanced],
    trainable=False,
)

# Architectures

def baseline_model(embedding_layer):
    baseline_model = Sequential()
    baseline_model.add(embedding_layer) # Now the output shape is (batch_size, □
    →max_length, embedding_dim)
    baseline_model.add(Bidirectional(LSTM(64)))
    baseline_model.add(Dense(1, activation="sigmoid"))
    baseline_model.compile(optimizer="adam", loss="binary_crossentropy", □
    →metrics=["accuracy"])
    return baseline_model

def model_add_BLSTM(embedding_layer):
    model1 = Sequential()
    model1.add(embedding_layer)
    model1.add(Bidirectional(LSTM(64, return_sequences=True)))
    model1.add(Bidirectional(LSTM(32)))
    model1.add(Dense(1, activation="sigmoid"))
    model1.compile(optimizer="adam", loss="binary_crossentropy", □
    →metrics=["accuracy"])
    return model1

```

Step 5 - Training Next cell trains the compiled model and returns the evaluation.

```

[ ]: seeds = [41, 72, 97]
    EPOCHS = 15
    BATCH_SIZE = 64

# Dictionary to store F1 results for each seed
results = {
    "Model_Base_U": [],
    "Model_Base_UA": [],
    "Model_Base_B": [],
    "Model_Base_BA": [],

```

```

}

# Dictionary to store model stuff
model_instances = {}
model_histories = {}

for seed in seeds:
    print(f"\n\n# ----- # ----- # Training with seed: {seed} # ----- #\n")
    print()

    tf.random.set_seed(seed)
    np.random.seed(seed)

    ### MODELS (RE)INITIALIZATION
    model_instances[f"Model_Base_U_seed_{seed}"] = baseline_model(embedding_layer)
    model_instances[f"Model_Base_UA_seed_{seed}"] = model_add_BLSTM(embedding_layer)

    model_instances[f"Model_Base_B_seed_{seed}"] = baseline_model(embedding_balanced_layer)
    model_instances[f"Model_Base_BA_seed_{seed}"] = model_add_BLSTM(embedding_balanced_layer)

    ### Unbalanced Section -----
    # --- Train Model_Base_U ---
    print("Training Model_Base_U")
    history_u = model_instances[f"Model_Base_U_seed_{seed}"].fit(
        train_padded,
        train_df["hard_label_task1"],
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_data=(val_padded, val_df["hard_label_task1"]),
        verbose=1,
    )
    model_histories[f"Model_Base_U_seed_{seed}"] = history_u

    # --- Train Model_Base_UA ---
    print("Training Model_Base_UA")
    history_u = model_instances[f"Model_Base_UA_seed_{seed}"].fit(
        train_padded,
        train_df["hard_label_task1"],
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_data=(val_padded, val_df["hard_label_task1"]),

```

```

        verbose=1,
    )
    model_histories[f"Model_Base_UA_seed_{seed}"] = history_ua

    ### Balanced Section -----
    # --- Train Model_Base_B ---
    print("Training Model_Base_B")
    history_b = model_instances[f"Model_Base_B_seed_{seed}"].fit(
        train_balanced_padded,
        train_df_balanced["hard_label_task1"],
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_data=(val_balanced_padded, val_df["hard_label_task1"]),
        verbose=1,
    )
    model_histories[f"Model_Base_B_seed_{seed}"] = history_b

    # --- Train Model_Base_BA ---
    print("Training Model_Base_BA")
    history_ba = model_instances[f"Model_Base_BA_seed_{seed}"].fit(
        train_balanced_padded,
        train_df_balanced["hard_label_task1"],
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_data=(val_balanced_padded, val_df["hard_label_task1"]),
        verbose=1,
    )
    model_histories[f"Model_Base_BA_seed_{seed}"] = history_ba

    ### Evaluation Section -----
    # --- Evaluate Models on Validation Set ---
    print(f"Evaluating models for seed: {seed}")

    # Predictions and F1 scores
    predictions_u = (model_instances[f"Model_Base_U_seed_{seed}"].
    ↪predict(val_padded) > 0.5).astype("int32")
    predictions_ua = (model_instances[f"Model_Base_UA_seed_{seed}"].
    ↪predict(val_padded) > 0.5).astype("int32")
    predictions_b = (model_instances[f"Model_Base_B_seed_{seed}"].
    ↪predict(val_balanced_padded) > 0.5).astype("int32")
    predictions_ba = (model_instances[f"Model_Base_BA_seed_{seed}"].
    ↪predict(val_balanced_padded) > 0.5).astype("int32")

    # Calculate F1 scores
    f1_u = f1_score(val_df["hard_label_task1"], predictions_u, average="macro")
    f1_ua = f1_score(val_df["hard_label_task1"], predictions_ua,
    ↪average="macro")

```

```

f1_b = f1_score(val_df["hard_label_task1"], predictions_b, average="macro")
f1_ba = f1_score(val_df["hard_label_task1"], predictions_ba,
↪average="macro")

# Store F1 Score results for later
results["Model_Base_U"].append(f1_u)
results["Model_Base_UA"].append(f1_ua)
results["Model_Base_B"].append(f1_b)
results["Model_Base_BA"].append(f1_ba)

print(f"Seed {seed} - F1 Scores:")
print(f"  Model_Base_U: {f1_u:.4f}")
print(f"  Model_Base_UA: {f1_ua:.4f}")
print(f"  Model_Base_B: {f1_b:.4f}")
print(f"  Model_Base_BA: {f1_ba:.4f}")

```

----- # ----- # Training with seed: 41 # ----- # -----

Training Model_Base_U

Epoch 1/15

45/45 5s 15ms/step -

accuracy: 0.5673 - loss: 0.6741 - val_accuracy: 0.6519 - val_loss: 0.6614

Epoch 2/15

45/45 0s 8ms/step -

accuracy: 0.6871 - loss: 0.6014 - val_accuracy: 0.6835 - val_loss: 0.5871

Epoch 3/15

45/45 0s 8ms/step -

accuracy: 0.7347 - loss: 0.5426 - val_accuracy: 0.7278 - val_loss: 0.5640

Epoch 4/15

45/45 0s 9ms/step -

accuracy: 0.7649 - loss: 0.5102 - val_accuracy: 0.7342 - val_loss: 0.5489

Epoch 5/15

45/45 0s 8ms/step -

accuracy: 0.7821 - loss: 0.4840 - val_accuracy: 0.7405 - val_loss: 0.5341

Epoch 6/15

45/45 0s 8ms/step -

accuracy: 0.7982 - loss: 0.4622 - val_accuracy: 0.7595 - val_loss: 0.5051

Epoch 7/15

45/45 0s 8ms/step -

accuracy: 0.8038 - loss: 0.4405 - val_accuracy: 0.7722 - val_loss: 0.5113

Epoch 8/15

45/45 0s 8ms/step -

accuracy: 0.8088 - loss: 0.4244 - val_accuracy: 0.7468 - val_loss: 0.5156

Epoch 9/15

45/45 0s 8ms/step -

accuracy: 0.8163 - loss: 0.4185 - val_accuracy: 0.7785 - val_loss: 0.5044

Epoch 10/15
45/45 0s 7ms/step -
accuracy: 0.8384 - loss: 0.3765 - val_accuracy: 0.7785 - val_loss: 0.5230
Epoch 11/15
45/45 0s 8ms/step -
accuracy: 0.8505 - loss: 0.3577 - val_accuracy: 0.7722 - val_loss: 0.5137
Epoch 12/15
45/45 0s 8ms/step -
accuracy: 0.8560 - loss: 0.3545 - val_accuracy: 0.7911 - val_loss: 0.4852
Epoch 13/15
45/45 0s 8ms/step -
accuracy: 0.8686 - loss: 0.3226 - val_accuracy: 0.7975 - val_loss: 0.5521
Epoch 14/15
45/45 0s 7ms/step -
accuracy: 0.8813 - loss: 0.3074 - val_accuracy: 0.7911 - val_loss: 0.6188
Epoch 15/15
45/45 0s 7ms/step -
accuracy: 0.9007 - loss: 0.2724 - val_accuracy: 0.7722 - val_loss: 0.5753
Training Model_Base_UA
Epoch 1/15
45/45 4s 22ms/step -
accuracy: 0.5871 - loss: 0.6668 - val_accuracy: 0.6646 - val_loss: 0.6227
Epoch 2/15
45/45 1s 12ms/step -
accuracy: 0.7032 - loss: 0.5756 - val_accuracy: 0.6582 - val_loss: 0.5991
Epoch 3/15
45/45 1s 12ms/step -
accuracy: 0.7338 - loss: 0.5431 - val_accuracy: 0.6835 - val_loss: 0.5676
Epoch 4/15
45/45 1s 12ms/step -
accuracy: 0.7705 - loss: 0.5063 - val_accuracy: 0.7468 - val_loss: 0.5318
Epoch 5/15
45/45 1s 12ms/step -
accuracy: 0.7772 - loss: 0.4829 - val_accuracy: 0.7468 - val_loss: 0.5129
Epoch 6/15
45/45 1s 12ms/step -
accuracy: 0.7986 - loss: 0.4587 - val_accuracy: 0.7658 - val_loss: 0.5025
Epoch 7/15
45/45 1s 12ms/step -
accuracy: 0.8154 - loss: 0.4341 - val_accuracy: 0.7658 - val_loss: 0.5006
Epoch 8/15
45/45 1s 12ms/step -
accuracy: 0.8184 - loss: 0.4132 - val_accuracy: 0.7595 - val_loss: 0.5519
Epoch 9/15
45/45 1s 13ms/step -
accuracy: 0.8286 - loss: 0.4032 - val_accuracy: 0.7722 - val_loss: 0.5053
Epoch 10/15
45/45 1s 12ms/step -

accuracy: 0.8421 - loss: 0.3801 - val_accuracy: 0.7848 - val_loss: 0.4937
 Epoch 11/15
 45/45 1s 12ms/step -
 accuracy: 0.8502 - loss: 0.3687 - val_accuracy: 0.7532 - val_loss: 0.5300
 Epoch 12/15
 45/45 1s 13ms/step -
 accuracy: 0.8427 - loss: 0.3734 - val_accuracy: 0.7911 - val_loss: 0.5101
 Epoch 13/15
 45/45 1s 12ms/step -
 accuracy: 0.8542 - loss: 0.3426 - val_accuracy: 0.7722 - val_loss: 0.5330
 Epoch 14/15
 45/45 1s 12ms/step -
 accuracy: 0.8908 - loss: 0.2960 - val_accuracy: 0.7595 - val_loss: 0.6692
 Epoch 15/15
 45/45 1s 12ms/step -
 accuracy: 0.8859 - loss: 0.2788 - val_accuracy: 0.7848 - val_loss: 0.6036
 Training Model_Base_B
 Epoch 1/15
 36/36 3s 16ms/step -
 accuracy: 0.5432 - loss: 0.6903 - val_accuracy: 0.5696 - val_loss: 0.6675
 Epoch 2/15
 36/36 0s 8ms/step -
 accuracy: 0.6562 - loss: 0.6282 - val_accuracy: 0.6646 - val_loss: 0.5797
 Epoch 3/15
 36/36 0s 8ms/step -
 accuracy: 0.7023 - loss: 0.5752 - val_accuracy: 0.6962 - val_loss: 0.5628
 Epoch 4/15
 36/36 0s 8ms/step -
 accuracy: 0.7366 - loss: 0.5491 - val_accuracy: 0.6962 - val_loss: 0.5521
 Epoch 5/15
 36/36 0s 8ms/step -
 accuracy: 0.7524 - loss: 0.5194 - val_accuracy: 0.7152 - val_loss: 0.5323
 Epoch 6/15
 36/36 0s 7ms/step -
 accuracy: 0.7756 - loss: 0.4918 - val_accuracy: 0.7278 - val_loss: 0.5204
 Epoch 7/15
 36/36 0s 8ms/step -
 accuracy: 0.7838 - loss: 0.4733 - val_accuracy: 0.7278 - val_loss: 0.5146
 Epoch 8/15
 36/36 0s 8ms/step -
 accuracy: 0.7984 - loss: 0.4483 - val_accuracy: 0.7468 - val_loss: 0.5132
 Epoch 9/15
 36/36 0s 8ms/step -
 accuracy: 0.8081 - loss: 0.4245 - val_accuracy: 0.7468 - val_loss: 0.5350
 Epoch 10/15
 36/36 0s 8ms/step -
 accuracy: 0.8191 - loss: 0.4062 - val_accuracy: 0.7342 - val_loss: 0.5473
 Epoch 11/15

36/36 0s 8ms/step -
 accuracy: 0.8183 - loss: 0.3999 - val_accuracy: 0.7468 - val_loss: 0.5209
 Epoch 12/15
 36/36 0s 8ms/step -
 accuracy: 0.8292 - loss: 0.3767 - val_accuracy: 0.7152 - val_loss: 0.5711
 Epoch 13/15
 36/36 0s 8ms/step -
 accuracy: 0.8227 - loss: 0.3962 - val_accuracy: 0.7658 - val_loss: 0.5168
 Epoch 14/15
 36/36 0s 8ms/step -
 accuracy: 0.8415 - loss: 0.3472 - val_accuracy: 0.7658 - val_loss: 0.4994
 Epoch 15/15
 36/36 0s 8ms/step -
 accuracy: 0.8608 - loss: 0.3219 - val_accuracy: 0.7405 - val_loss: 0.5223
 Training Model_Base_BA
 Epoch 1/15
 36/36 4s 26ms/step -
 accuracy: 0.5622 - loss: 0.6792 - val_accuracy: 0.6899 - val_loss: 0.5974
 Epoch 2/15
 36/36 1s 14ms/step -
 accuracy: 0.6756 - loss: 0.5959 - val_accuracy: 0.7089 - val_loss: 0.5704
 Epoch 3/15
 36/36 0s 13ms/step -
 accuracy: 0.7246 - loss: 0.5533 - val_accuracy: 0.7342 - val_loss: 0.5507
 Epoch 4/15
 36/36 1s 14ms/step -
 accuracy: 0.7438 - loss: 0.5247 - val_accuracy: 0.7532 - val_loss: 0.5394
 Epoch 5/15
 36/36 0s 13ms/step -
 accuracy: 0.7695 - loss: 0.4998 - val_accuracy: 0.7658 - val_loss: 0.5440
 Epoch 6/15
 36/36 1s 13ms/step -
 accuracy: 0.7867 - loss: 0.4726 - val_accuracy: 0.7722 - val_loss: 0.5061
 Epoch 7/15
 36/36 1s 14ms/step -
 accuracy: 0.8013 - loss: 0.4391 - val_accuracy: 0.7215 - val_loss: 0.5825
 Epoch 8/15
 36/36 0s 13ms/step -
 accuracy: 0.8005 - loss: 0.4515 - val_accuracy: 0.7532 - val_loss: 0.5391
 Epoch 9/15
 36/36 0s 13ms/step -
 accuracy: 0.8152 - loss: 0.4233 - val_accuracy: 0.7658 - val_loss: 0.5306
 Epoch 10/15
 36/36 0s 13ms/step -
 accuracy: 0.8317 - loss: 0.3979 - val_accuracy: 0.7468 - val_loss: 0.5433
 Epoch 11/15
 36/36 0s 13ms/step -
 accuracy: 0.8500 - loss: 0.3770 - val_accuracy: 0.7595 - val_loss: 0.4979

```

Epoch 12/15
36/36          1s 14ms/step -
accuracy: 0.8187 - loss: 0.4089 - val_accuracy: 0.7658 - val_loss: 0.4856
Epoch 13/15
36/36          0s 13ms/step -
accuracy: 0.8480 - loss: 0.3558 - val_accuracy: 0.7658 - val_loss: 0.4898
Epoch 14/15
36/36          0s 13ms/step -
accuracy: 0.8788 - loss: 0.3081 - val_accuracy: 0.7848 - val_loss: 0.5247
Epoch 15/15
36/36          0s 13ms/step -
accuracy: 0.8800 - loss: 0.2846 - val_accuracy: 0.7785 - val_loss: 0.5123
Evaluating models for seed: 41
5/5           0s 47ms/step
5/5           1s 83ms/step

```

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7859948c51b0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

```

1/5           0s 180ms/step

```

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7859948c51b0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

```

5/5           0s 49ms/step
5/5           1s 83ms/step

```

Seed 41 - F1 Scores:

```

Model_Base_U: 0.7582
Model_Base_UA: 0.7649
Model_Base_B: 0.7358
Model_Base_BA: 0.7728

```

```

# ----- # ----- # Training with seed: 72 # ----- # ----- #

```


Training Model_Base_U

Epoch 1/15

45/45 3s 14ms/step -

accuracy: 0.6080 - loss: 0.6578 - val_accuracy: 0.7152 - val_loss: 0.6058

Epoch 2/15

45/45 0s 7ms/step -

accuracy: 0.7180 - loss: 0.5598 - val_accuracy: 0.7025 - val_loss: 0.5890

Epoch 3/15

45/45 0s 8ms/step -

accuracy: 0.7385 - loss: 0.5281 - val_accuracy: 0.7278 - val_loss: 0.5865

Epoch 4/15

45/45 0s 8ms/step -

accuracy: 0.7544 - loss: 0.5094 - val_accuracy: 0.7342 - val_loss: 0.5555

Epoch 5/15

45/45 0s 8ms/step -

accuracy: 0.7661 - loss: 0.4926 - val_accuracy: 0.7468 - val_loss: 0.5373

Epoch 6/15

45/45 0s 8ms/step -

accuracy: 0.7766 - loss: 0.4647 - val_accuracy: 0.7532 - val_loss: 0.5378

Epoch 7/15

45/45 0s 8ms/step -

accuracy: 0.7958 - loss: 0.4412 - val_accuracy: 0.7658 - val_loss: 0.5193

Epoch 8/15

45/45 0s 7ms/step -

accuracy: 0.8140 - loss: 0.4172 - val_accuracy: 0.7532 - val_loss: 0.5124

Epoch 9/15

45/45 0s 7ms/step -

accuracy: 0.8351 - loss: 0.3915 - val_accuracy: 0.7658 - val_loss: 0.5505

Epoch 10/15

45/45 0s 8ms/step -

accuracy: 0.8352 - loss: 0.3826 - val_accuracy: 0.7532 - val_loss: 0.5603

Epoch 11/15

45/45 0s 8ms/step -

accuracy: 0.8466 - loss: 0.3599 - val_accuracy: 0.7785 - val_loss: 0.5222

Epoch 12/15

45/45 0s 7ms/step -

accuracy: 0.8585 - loss: 0.3357 - val_accuracy: 0.7785 - val_loss: 0.5178

Epoch 13/15

45/45 0s 8ms/step -

accuracy: 0.8669 - loss: 0.3182 - val_accuracy: 0.7848 - val_loss: 0.5281

Epoch 14/15

45/45 0s 8ms/step -

accuracy: 0.8869 - loss: 0.2971 - val_accuracy: 0.7975 - val_loss: 0.5422

Epoch 15/15

45/45 0s 8ms/step -

accuracy: 0.8905 - loss: 0.2717 - val_accuracy: 0.7848 - val_loss: 0.5985

Training Model_Base_UA

Epoch 1/15
45/45 5s 23ms/step -
accuracy: 0.5902 - loss: 0.6586 - val_accuracy: 0.7278 - val_loss: 0.5975
Epoch 2/15
45/45 1s 13ms/step -
accuracy: 0.7110 - loss: 0.5707 - val_accuracy: 0.6962 - val_loss: 0.5809
Epoch 3/15
45/45 1s 12ms/step -
accuracy: 0.7348 - loss: 0.5337 - val_accuracy: 0.7025 - val_loss: 0.5762
Epoch 4/15
45/45 1s 12ms/step -
accuracy: 0.7608 - loss: 0.5098 - val_accuracy: 0.7278 - val_loss: 0.5569
Epoch 5/15
45/45 1s 12ms/step -
accuracy: 0.7657 - loss: 0.4880 - val_accuracy: 0.7658 - val_loss: 0.5351
Epoch 6/15
45/45 1s 13ms/step -
accuracy: 0.7875 - loss: 0.4648 - val_accuracy: 0.7658 - val_loss: 0.5068
Epoch 7/15
45/45 1s 13ms/step -
accuracy: 0.7907 - loss: 0.4393 - val_accuracy: 0.7848 - val_loss: 0.5163
Epoch 8/15
45/45 1s 12ms/step -
accuracy: 0.8227 - loss: 0.4053 - val_accuracy: 0.7722 - val_loss: 0.5117
Epoch 9/15
45/45 1s 13ms/step -
accuracy: 0.8481 - loss: 0.3700 - val_accuracy: 0.7848 - val_loss: 0.5429
Epoch 10/15
45/45 1s 12ms/step -
accuracy: 0.8699 - loss: 0.3278 - val_accuracy: 0.7658 - val_loss: 0.5168
Epoch 11/15
45/45 1s 12ms/step -
accuracy: 0.8765 - loss: 0.3091 - val_accuracy: 0.7722 - val_loss: 0.5461
Epoch 12/15
45/45 1s 12ms/step -
accuracy: 0.8601 - loss: 0.3275 - val_accuracy: 0.8165 - val_loss: 0.5085
Epoch 13/15
45/45 1s 12ms/step -
accuracy: 0.8780 - loss: 0.2918 - val_accuracy: 0.7405 - val_loss: 0.7103
Epoch 14/15
45/45 1s 12ms/step -
accuracy: 0.8899 - loss: 0.2701 - val_accuracy: 0.7658 - val_loss: 0.6588
Epoch 15/15
45/45 1s 12ms/step -
accuracy: 0.9181 - loss: 0.2325 - val_accuracy: 0.7658 - val_loss: 0.7241
Training Model_Base_B
Epoch 1/15
36/36 3s 17ms/step -

```

accuracy: 0.5560 - loss: 0.6788 - val_accuracy: 0.7215 - val_loss: 0.5987
Epoch 2/15
36/36          0s 9ms/step -
accuracy: 0.6858 - loss: 0.6019 - val_accuracy: 0.7405 - val_loss: 0.5673
Epoch 3/15
36/36          0s 8ms/step -
accuracy: 0.7268 - loss: 0.5545 - val_accuracy: 0.7468 - val_loss: 0.5382
Epoch 4/15
36/36          0s 8ms/step -
accuracy: 0.7460 - loss: 0.5265 - val_accuracy: 0.7405 - val_loss: 0.5249
Epoch 5/15
36/36          0s 8ms/step -
accuracy: 0.7706 - loss: 0.5024 - val_accuracy: 0.7468 - val_loss: 0.5150
Epoch 6/15
36/36          0s 8ms/step -
accuracy: 0.7844 - loss: 0.4790 - val_accuracy: 0.7595 - val_loss: 0.5090
Epoch 7/15
36/36          0s 8ms/step -
accuracy: 0.7990 - loss: 0.4536 - val_accuracy: 0.7405 - val_loss: 0.5119
Epoch 8/15
36/36          0s 8ms/step -
accuracy: 0.8134 - loss: 0.4352 - val_accuracy: 0.7468 - val_loss: 0.5033
Epoch 9/15
36/36          0s 8ms/step -
accuracy: 0.8277 - loss: 0.4059 - val_accuracy: 0.7785 - val_loss: 0.5314
Epoch 10/15
36/36          0s 9ms/step -
accuracy: 0.8302 - loss: 0.3940 - val_accuracy: 0.7532 - val_loss: 0.5310
Epoch 11/15
36/36          0s 10ms/step -
accuracy: 0.8317 - loss: 0.3855 - val_accuracy: 0.7848 - val_loss: 0.4957
Epoch 12/15
36/36          0s 9ms/step -
accuracy: 0.8521 - loss: 0.3534 - val_accuracy: 0.7468 - val_loss: 0.5751
Epoch 13/15
36/36          0s 8ms/step -
accuracy: 0.8443 - loss: 0.3646 - val_accuracy: 0.7658 - val_loss: 0.5109
Epoch 14/15
36/36          0s 9ms/step -
accuracy: 0.8536 - loss: 0.3577 - val_accuracy: 0.7658 - val_loss: 0.6031
Epoch 15/15
36/36          0s 9ms/step -
accuracy: 0.8736 - loss: 0.3155 - val_accuracy: 0.7911 - val_loss: 0.5912
Training Model_Base_BA
Epoch 1/15
36/36          4s 26ms/step -
accuracy: 0.5857 - loss: 0.6724 - val_accuracy: 0.7152 - val_loss: 0.5855
Epoch 2/15

```

```

36/36          0s 13ms/step -
accuracy: 0.6958 - loss: 0.5906 - val_accuracy: 0.7215 - val_loss: 0.5434
Epoch 3/15
36/36          0s 13ms/step -
accuracy: 0.7246 - loss: 0.5558 - val_accuracy: 0.7278 - val_loss: 0.5355
Epoch 4/15
36/36          0s 13ms/step -
accuracy: 0.7517 - loss: 0.5205 - val_accuracy: 0.7595 - val_loss: 0.5240
Epoch 5/15
36/36          0s 13ms/step -
accuracy: 0.7726 - loss: 0.5036 - val_accuracy: 0.7658 - val_loss: 0.5264
Epoch 6/15
36/36          0s 13ms/step -
accuracy: 0.7806 - loss: 0.4787 - val_accuracy: 0.7785 - val_loss: 0.5075
Epoch 7/15
36/36          0s 13ms/step -
accuracy: 0.8042 - loss: 0.4429 - val_accuracy: 0.8101 - val_loss: 0.5046
Epoch 8/15
36/36          0s 13ms/step -
accuracy: 0.8212 - loss: 0.4177 - val_accuracy: 0.7975 - val_loss: 0.5079
Epoch 9/15
36/36          1s 14ms/step -
accuracy: 0.8447 - loss: 0.3882 - val_accuracy: 0.7911 - val_loss: 0.5226
Epoch 10/15
36/36          1s 14ms/step -
accuracy: 0.8305 - loss: 0.3974 - val_accuracy: 0.7658 - val_loss: 0.4970
Epoch 11/15
36/36          1s 14ms/step -
accuracy: 0.8549 - loss: 0.3569 - val_accuracy: 0.7342 - val_loss: 0.5854
Epoch 12/15
36/36          1s 15ms/step -
accuracy: 0.8538 - loss: 0.3588 - val_accuracy: 0.7152 - val_loss: 0.6196
Epoch 13/15
36/36          1s 14ms/step -
accuracy: 0.8466 - loss: 0.3675 - val_accuracy: 0.7658 - val_loss: 0.5036
Epoch 14/15
36/36          0s 13ms/step -
accuracy: 0.8712 - loss: 0.3280 - val_accuracy: 0.7532 - val_loss: 0.5061
Epoch 15/15
36/36          1s 13ms/step -
accuracy: 0.8942 - loss: 0.2651 - val_accuracy: 0.7722 - val_loss: 0.5984
Evaluating models for seed: 72
5/5           0s 47ms/step
5/5           1s 84ms/step
5/5           0s 46ms/step
5/5           1s 82ms/step
Seed 72 - F1 Scores:
  Model_Base_U: 0.7730

```

Model_Base_UA: 0.7279
Model_Base_B: 0.7761
Model_Base_BA: 0.7648

----- # ----- # Training with seed: 97 # ----- # -----

Training Model_Base_U

Epoch 1/15

45/45 3s 14ms/step -

accuracy: 0.5892 - loss: 0.6684 - val_accuracy: 0.7089 - val_loss: 0.5690

Epoch 2/15

45/45 0s 8ms/step -

accuracy: 0.7117 - loss: 0.5614 - val_accuracy: 0.7532 - val_loss: 0.5313

Epoch 3/15

45/45 0s 8ms/step -

accuracy: 0.7368 - loss: 0.5313 - val_accuracy: 0.7342 - val_loss: 0.5301

Epoch 4/15

45/45 0s 8ms/step -

accuracy: 0.7480 - loss: 0.5176 - val_accuracy: 0.7405 - val_loss: 0.5433

Epoch 5/15

45/45 0s 7ms/step -

accuracy: 0.7659 - loss: 0.4972 - val_accuracy: 0.7848 - val_loss: 0.5181

Epoch 6/15

45/45 0s 8ms/step -

accuracy: 0.7805 - loss: 0.4759 - val_accuracy: 0.7785 - val_loss: 0.5352

Epoch 7/15

45/45 0s 8ms/step -

accuracy: 0.7941 - loss: 0.4560 - val_accuracy: 0.8101 - val_loss: 0.5000

Epoch 8/15

45/45 0s 7ms/step -

accuracy: 0.8161 - loss: 0.4222 - val_accuracy: 0.7975 - val_loss: 0.5261

Epoch 9/15

45/45 0s 8ms/step -

accuracy: 0.8290 - loss: 0.4013 - val_accuracy: 0.7975 - val_loss: 0.4833

Epoch 10/15

45/45 0s 8ms/step -

accuracy: 0.8439 - loss: 0.3841 - val_accuracy: 0.8228 - val_loss: 0.4521

Epoch 11/15

45/45 0s 8ms/step -

accuracy: 0.8552 - loss: 0.3645 - val_accuracy: 0.8038 - val_loss: 0.4691

Epoch 12/15

45/45 0s 8ms/step -

accuracy: 0.8447 - loss: 0.3633 - val_accuracy: 0.8101 - val_loss: 0.4603

Epoch 13/15

45/45 0s 8ms/step -

accuracy: 0.8470 - loss: 0.3563 - val_accuracy: 0.7405 - val_loss: 0.6462

Epoch 14/15

45/45 0s 8ms/step -
 accuracy: 0.8489 - loss: 0.3479 - val_accuracy: 0.6962 - val_loss: 0.6892
 Epoch 15/15
 45/45 0s 8ms/step -
 accuracy: 0.8502 - loss: 0.3427 - val_accuracy: 0.7848 - val_loss: 0.5513
 Training Model_Base-UA
 Epoch 1/15
 45/45 4s 23ms/step -
 accuracy: 0.6100 - loss: 0.6560 - val_accuracy: 0.7089 - val_loss: 0.5870
 Epoch 2/15
 45/45 1s 13ms/step -
 accuracy: 0.7166 - loss: 0.5679 - val_accuracy: 0.7089 - val_loss: 0.5684
 Epoch 3/15
 45/45 1s 13ms/step -
 accuracy: 0.7412 - loss: 0.5313 - val_accuracy: 0.7342 - val_loss: 0.5694
 Epoch 4/15
 45/45 1s 13ms/step -
 accuracy: 0.7653 - loss: 0.4988 - val_accuracy: 0.7405 - val_loss: 0.5548
 Epoch 5/15
 45/45 1s 12ms/step -
 accuracy: 0.7853 - loss: 0.4807 - val_accuracy: 0.7215 - val_loss: 0.5970
 Epoch 6/15
 45/45 1s 12ms/step -
 accuracy: 0.7782 - loss: 0.4718 - val_accuracy: 0.7468 - val_loss: 0.5760
 Epoch 7/15
 45/45 1s 12ms/step -
 accuracy: 0.8028 - loss: 0.4363 - val_accuracy: 0.7848 - val_loss: 0.5149
 Epoch 8/15
 45/45 1s 13ms/step -
 accuracy: 0.8242 - loss: 0.4096 - val_accuracy: 0.7595 - val_loss: 0.5582
 Epoch 9/15
 45/45 1s 12ms/step -
 accuracy: 0.8390 - loss: 0.3845 - val_accuracy: 0.7722 - val_loss: 0.5551
 Epoch 10/15
 45/45 1s 13ms/step -
 accuracy: 0.8636 - loss: 0.3510 - val_accuracy: 0.7848 - val_loss: 0.4845
 Epoch 11/15
 45/45 1s 12ms/step -
 accuracy: 0.8580 - loss: 0.3461 - val_accuracy: 0.7848 - val_loss: 0.4772
 Epoch 12/15
 45/45 1s 12ms/step -
 accuracy: 0.8551 - loss: 0.3374 - val_accuracy: 0.7848 - val_loss: 0.5282
 Epoch 13/15
 45/45 1s 14ms/step -
 accuracy: 0.8843 - loss: 0.3025 - val_accuracy: 0.7278 - val_loss: 0.7022
 Epoch 14/15
 45/45 1s 14ms/step -
 accuracy: 0.9118 - loss: 0.2411 - val_accuracy: 0.7532 - val_loss: 0.9281

Epoch 15/15
45/45 1s 14ms/step -
accuracy: 0.9129 - loss: 0.2284 - val_accuracy: 0.8038 - val_loss: 0.6288
Training Model_Base_B

Epoch 1/15
36/36 3s 16ms/step -
accuracy: 0.5509 - loss: 0.6874 - val_accuracy: 0.6266 - val_loss: 0.6231

Epoch 2/15
36/36 0s 8ms/step -
accuracy: 0.6816 - loss: 0.6222 - val_accuracy: 0.5886 - val_loss: 0.6453

Epoch 3/15
36/36 0s 8ms/step -
accuracy: 0.7127 - loss: 0.5844 - val_accuracy: 0.7025 - val_loss: 0.5722

Epoch 4/15
36/36 0s 8ms/step -
accuracy: 0.7243 - loss: 0.5528 - val_accuracy: 0.7215 - val_loss: 0.5556

Epoch 5/15
36/36 0s 8ms/step -
accuracy: 0.7442 - loss: 0.5351 - val_accuracy: 0.7215 - val_loss: 0.5520

Epoch 6/15
36/36 0s 8ms/step -
accuracy: 0.7499 - loss: 0.5181 - val_accuracy: 0.7468 - val_loss: 0.5425

Epoch 7/15
36/36 0s 8ms/step -
accuracy: 0.7680 - loss: 0.4964 - val_accuracy: 0.7152 - val_loss: 0.5466

Epoch 8/15
36/36 0s 8ms/step -
accuracy: 0.7935 - loss: 0.4720 - val_accuracy: 0.7278 - val_loss: 0.5410

Epoch 9/15
36/36 0s 8ms/step -
accuracy: 0.8106 - loss: 0.4463 - val_accuracy: 0.7342 - val_loss: 0.5536

Epoch 10/15
36/36 0s 8ms/step -
accuracy: 0.8197 - loss: 0.4225 - val_accuracy: 0.7342 - val_loss: 0.5830

Epoch 11/15
36/36 0s 8ms/step -
accuracy: 0.8364 - loss: 0.4002 - val_accuracy: 0.7278 - val_loss: 0.6132

Epoch 12/15
36/36 0s 8ms/step -
accuracy: 0.8076 - loss: 0.4328 - val_accuracy: 0.7722 - val_loss: 0.5012

Epoch 13/15
36/36 0s 8ms/step -
accuracy: 0.8216 - loss: 0.4144 - val_accuracy: 0.7595 - val_loss: 0.5298

Epoch 14/15
36/36 0s 8ms/step -
accuracy: 0.8381 - loss: 0.3884 - val_accuracy: 0.7595 - val_loss: 0.5184

Epoch 15/15
36/36 0s 8ms/step -

```

accuracy: 0.8704 - loss: 0.3345 - val_accuracy: 0.7785 - val_loss: 0.5459
Training Model_Base_BA
Epoch 1/15
36/36          4s 25ms/step -
accuracy: 0.5620 - loss: 0.6800 - val_accuracy: 0.6519 - val_loss: 0.6130
Epoch 2/15
36/36          0s 13ms/step -
accuracy: 0.6945 - loss: 0.5933 - val_accuracy: 0.7532 - val_loss: 0.5714
Epoch 3/15
36/36          0s 13ms/step -
accuracy: 0.7279 - loss: 0.5549 - val_accuracy: 0.7278 - val_loss: 0.5531
Epoch 4/15
36/36          0s 13ms/step -
accuracy: 0.7500 - loss: 0.5301 - val_accuracy: 0.7532 - val_loss: 0.5510
Epoch 5/15
36/36          0s 13ms/step -
accuracy: 0.7680 - loss: 0.5093 - val_accuracy: 0.7342 - val_loss: 0.5444
Epoch 6/15
36/36          0s 13ms/step -
accuracy: 0.7781 - loss: 0.4897 - val_accuracy: 0.7468 - val_loss: 0.5305
Epoch 7/15
36/36          0s 12ms/step -
accuracy: 0.7959 - loss: 0.4715 - val_accuracy: 0.7532 - val_loss: 0.5127
Epoch 8/15
36/36          0s 12ms/step -
accuracy: 0.7879 - loss: 0.4647 - val_accuracy: 0.7722 - val_loss: 0.5161
Epoch 9/15
36/36          0s 13ms/step -
accuracy: 0.8003 - loss: 0.4394 - val_accuracy: 0.7911 - val_loss: 0.5224
Epoch 10/15
36/36          0s 13ms/step -
accuracy: 0.8304 - loss: 0.3976 - val_accuracy: 0.8165 - val_loss: 0.5008
Epoch 11/15
36/36          0s 13ms/step -
accuracy: 0.8475 - loss: 0.3588 - val_accuracy: 0.7848 - val_loss: 0.5263
Epoch 12/15
36/36          0s 13ms/step -
accuracy: 0.8698 - loss: 0.3156 - val_accuracy: 0.7911 - val_loss: 0.6221
Epoch 13/15
36/36          0s 13ms/step -
accuracy: 0.8485 - loss: 0.3419 - val_accuracy: 0.7785 - val_loss: 0.5233
Epoch 14/15
36/36          0s 13ms/step -
accuracy: 0.8869 - loss: 0.2800 - val_accuracy: 0.7911 - val_loss: 0.5628
Epoch 15/15
36/36          0s 13ms/step -
accuracy: 0.9103 - loss: 0.2354 - val_accuracy: 0.7722 - val_loss: 0.6268
Evaluating models for seed: 97

```



```

5/5          0s 47ms/step
5/5          1s 84ms/step
5/5          0s 48ms/step
5/5          1s 85ms/step

```

Seed 97 - F1 Scores:

```

Model_Base_U: 0.7629
Model_Base_UA: 0.7881
Model_Base_B: 0.7770
Model_Base_BA: 0.7511

```

Let's check out the output of training for the models:

```

[ ]: # --- Calculate Average F1 Scores Across Seeds for Validation set ---
average_f1_scores = {model: np.mean(scores) for model, scores in results.
    ↪ items()}
best_model = max(average_f1_scores, key=average_f1_scores.get)

print("\nAverage Macro-F1 scores across seeds on Validation set:")
for model_name, score in average_f1_scores.items():
    print(f"  {model_name}: {score:.4f}")

print(f"\nBest model based on average Macro-F1 score: {best_model}")

```

Average Macro-F1 scores across seeds on Validation set:

```

Model_Base_U: 0.7647
Model_Base_UA: 0.7603
Model_Base_B: 0.7630
Model_Base_BA: 0.7629

```

Best model based on average Macro-F1 score: Model_Base_U

The above results are the reported results about the validation accuracy.

Following cell goes through the histories of models and plots the learning curve. This code snippet is taken from the [Geeks for Geeks](#). However, before doing the visualization, we will calculate an average on the metrics of each model on each seeds. In other words, we average the different accuracies achieved by one model on different seeds. Then, we will use this average accuracy to demonstrate the learning curve. This demonstration is done in the error analysis section.

```

[ ]: average_acc_model_base = {
    "Model_Base_U": {"acc": [], "val_acc": [], "loss": [], "val_loss": []},
    "Model_Base_UA": {"acc": [], "val_acc": [], "loss": [], "val_loss": []},
    "Model_Base_B": {"acc": [], "val_acc": [], "loss": [], "val_loss": []},
    "Model_Base_BA": {"acc": [], "val_acc": [], "loss": [], "val_loss": []},
}

# Merging on model name
for model, history in model_histories.items():

```

```

model_name = model.split("_seed_")[0]
history_dict = history.history

# Seperating validation and training accuracy
acc = history_dict["accuracy"]
val_acc = history_dict["val_accuracy"]

# Seperating validation and training loss
loss = history_dict["loss"]
val_loss = history_dict["val_loss"]

average_acc_model_base[model_name] ["acc"] .append(acc)
average_acc_model_base[model_name] ["val_acc"] .append(val_acc)
average_acc_model_base[model_name] ["loss"] .append(loss)
average_acc_model_base[model_name] ["val_loss"] .append(val_loss)

# Averaging the metrics on seeds
for model_name, metric_dict in average_acc_model_base.items():
    for metric_name, metric_list in metric_dict.items():
        average_acc_model_base[model_name] [metric_name] = np.mean(metric_list,
↪axis=0)

```

Step 6 - Test Evaluation Let's test the models on the test data:

```

[ ]: LSTM_test_predictions = {}

for i, (model_name, model) in enumerate(model_instances.items()):
    print(model_name, end=" : ")

    if (i + 1) % 4 in (3, 0):
        test_data = test_balanced_padded
    else:
        test_data = test_padded

    test_predictions = (model.predict(test_data) > 0.5).astype("int32")
    test_f1 = f1_score(test_df["hard_label_task1"], test_predictions,
↪average="macro")

    LSTM_test_predictions[model_name] = [test_predictions, test_f1]

# Calculating the averages of the accuracies of models on test sets across the
↪seeds:
average_LSTM_test_predictions = {
    "Model_Base_U": [],
    "Model_Base-UA": [],
    "Model_Base_B": [],

```

```

    "Model_Base_BA": [],
}
for model_name in average_LSTM_test_predictions.keys():
    scores = []
    for seed in seeds:
        scores.append(LSTM_test_predictions[f"{model_name}_seed_{seed}"][1])
    average_LSTM_test_predictions[model_name] = np.mean(scores)

best_model_test = max(average_LSTM_test_predictions,
    ↪key=average_LSTM_test_predictions.get)

print("\nAverage Macro-F1 scores across seeds on Test set:")
for model_name, score in average_LSTM_test_predictions.items():
    print(f"  {model_name}: {score:.4f}")

print(f"\nBest model based on average Macro-F1 score: {best_model_test}")

```

```

9/9          0s 10ms/step
9/9          0s 17ms/step
9/9          0s 10ms/step
9/9          0s 17ms/step
9/9          0s 10ms/step
9/9          0s 17ms/step
9/9          0s 10ms/step
9/9          0s 17ms/step
9/9          0s 3ms/step
9/9          0s 4ms/step
9/9          0s 3ms/step
9/9          0s 4ms/step

```

Average Macro-F1 scores across seeds on Test set:

```

Model_Base_U: 0.7172
Model_Base_UA: 0.7173
Model_Base_B: 0.6929
Model_Base_BA: 0.6894

```

Best model based on average Macro-F1 score: Model_Base_UA

The above test results are reported in the report file as the test F1 score.

In the next cell, we are applying the majority voting to the predictions of the test set based on the seeds to also have a one prediction for it (same idea as averaging).

```

[ ]: LSTM_majority_voting_predictions = {}

def pred_voting(list_of_predictions):
    count = Counter([each_pred[0] for each_pred in list_of_predictions])

```

```

        if count[0] > count[1]:
            return 0
        elif count[1] > count[0]:
            return 1
        else:
            return 0

for model_name in set([name.split("_seed_")[0] for name in
↳LSTM_test_predictions.keys()]):
    predictions = []
    for seed in seeds:
        predictions.
↳append(LSTM_test_predictions[f"{model_name}_seed_{seed}"][0])

    # majority voting on that specific predictions
    majority_vote = []
    list_of_pred_for_that_idx = []
    for voting_for_index in range(len(predictions[0])):
        list_of_pred_for_that_idx = [pred_of_each_seed[voting_for_index] for
↳pred_of_each_seed in predictions]
        majority_vote.append(pred_voting(list_of_pred_for_that_idx))

    LSTM_majority_voting_predictions[model_name] = majority_vote

```

Transformer Model

In this section, we will use a transformer model specifically trained for hate speech detection, namely [Twitter-roBERTa-base for Hate Speech Detection](#).

Step 1 - Importing the Transformer and Preprocessing

```

[ ]: train_data = DS.from_pandas(train_df)
    train_data_balanced = DS.from_pandas(train_df_balanced)

    val_data = DS.from_pandas(val_df)
    test_data = DS.from_pandas(test_df)

```

We need to tokenize the text with transformer's tokenization!

```

[ ]: %%capture
    # Load the Tokenizer
    task = "hate"
    MODEL_NAME = f"cardiffnlp/twitter-roberta-base-{task}"

    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

    def preprocess_text(texts):

```

```

    return tokenizer(texts["tweet"], truncation=True, max_length=max_length) #
↪ Same Max length as the BLSTMs

```

```

train_data = train_data.map(preprocess_text, batched=True)
train_data_balanced = train_data_balanced.map(preprocess_text, batched=True)
val_data = val_data.map(preprocess_text, batched=True)
test_data = test_data.map(preprocess_text, batched=True)

```

```

[ ]: train_data = train_data.rename_column("hard_label_task1", "label")
train_data_balanced = train_data_balanced.rename_column("hard_label_task1",
↪ "label")
val_data = val_data.rename_column("hard_label_task1", "label")
test_data = test_data.rename_column("hard_label_task1", "label")

```

Step 2 - Defining the Model and Metrics It receives a batch of (input_ids, attention_mask, token_type_ids, label)

```

[ ]: data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2, id2label={0: "NO", 1: "YES"}, label2id={"NO": 0,
↪ "YES": 1}
)

```

In the following, we will use the Trainer API to do the training.

```

[ ]: acc_metric = evaluate.load("accuracy")
f1_metric = evaluate.load("f1")

def compute_metrics(output_info):
    predictions, labels = output_info
    predictions = np.argmax(predictions, axis=-1)

    f1 = f1_metric.compute(predictions=predictions, references=labels,
↪ average="macro")
    acc = acc_metric.compute(predictions=predictions, references=labels)
    return {**f1, **acc}

```

Step 3 - Training Training the initialized models with different seeds. We are also saving the models for later when we are calling them on the test sets.

NLP Tutorials Notebooks was fruitful for these sections.

```

[ ]: EPOCHS = 5
seeds = [512, 1024, 2048]
transformer_results_path = "Transformer_results"

```

```

# Directory to save results
results_dir = Path(transformer_results_path)
results_dir.mkdir(exist_ok=True)

# Results of each seed and dataset
log_history = []

datasets = {"unbalanced": train_data, "balanced": train_data_balanced}

for seed in seeds:
    set_seed(seed)
    print(f"Training with seed: {seed}")

    for dataset_name, dataset in datasets.items():
        print(f"Training on {dataset_name} dataset")

        # ----- Arguments settings
        output_dir = results_dir / f"{dataset_name}_seed_{seed}"
        output_dir.mkdir(exist_ok=True)

        training_args = TrainingArguments(
            output_dir=str(output_dir),
            learning_rate=2e-5,
            per_device_train_batch_size=8,
            per_device_eval_batch_size=8,
            num_train_epochs=EPOCHS,
            weight_decay=0.01,
            eval_strategy="epoch",
            save_strategy="epoch",
            load_best_model_at_end=True,
            metric_for_best_model="eval_accuracy",
            seed=seed, # Updating Every Seed
            report_to="none",
        )

        # (Re)Initialize Trainer On the new seed and data
        trainer = Trainer(
            model=model,
            args=training_args,
            train_dataset=dataset, # Updating dataset
            eval_dataset=val_data,
            data_collator=data_collator,
            compute_metrics=compute_metrics,
        )
        train_output = trainer.train()

        # Save the model and tokenizer for test section

```

```

trainer.model.save_pretrained(str(output_dir / "model"))
tokenizer.save_pretrained(str(output_dir / "tokenizer"))

# Evaluate the model
eval_results = trainer.evaluate()
print(f"Dataset: {dataset_name} -> Results for seed {seed}:")
↪{eval_results}")

# Save log history
log_hist = trainer.state.log_history
for log in log_hist:
    if "epoch" in log:
        log_history.append(
            {
                "seed": seed,
                "dataset": dataset_name,
                "epoch": log["epoch"],
                "log": log,
            }
        )

```

Training with seed: 512

Training on unbalanced dataset

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Dataset: unbalanced -> Results for seed 512: {'eval_loss': 0.32183462381362915, 'eval_f1': 0.8680033416875522, 'eval_accuracy': 0.8734177215189873, 'eval_runtime': 0.2548, 'eval_samples_per_second': 619.977, 'eval_steps_per_second': 78.478, 'epoch': 5.0}

Training on balanced dataset

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Dataset: balanced -> Results for seed 512: {'eval_loss': 0.49702438712120056, 'eval_f1': 0.8829051383399209, 'eval_accuracy': 0.8860759493670886, 'eval_runtime': 0.2578, 'eval_samples_per_second': 612.851, 'eval_steps_per_second': 77.576, 'epoch': 5.0}

Training with seed: 1024

Training on unbalanced dataset

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Dataset: unbalanced -> Results for seed 1024: {'eval_loss': 1.2444831132888794, 'eval_f1': 0.8531350346459354, 'eval_accuracy': 0.8607594936708861, 'eval_runtime': 0.2479, 'eval_samples_per_second': 637.308,

```

'eval_steps_per_second': 80.672, 'epoch': 5.0}
Training on balanced dataset

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Dataset: balanced -> Results for seed 1024: {'eval_loss': 1.3121387958526611,
'eval_f1': 0.8610261780104712, 'eval_accuracy': 0.8670886075949367,
'eval_runtime': 0.2621, 'eval_samples_per_second': 602.734,
'eval_steps_per_second': 76.295, 'epoch': 5.0}
Training with seed: 2048
Training on unbalanced dataset

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Dataset: unbalanced -> Results for seed 2048: {'eval_loss': 1.181212067604065,
'eval_f1': 0.8664863951326687, 'eval_accuracy': 0.8734177215189873,
'eval_runtime': 0.2502, 'eval_samples_per_second': 631.531,
'eval_steps_per_second': 79.941, 'epoch': 5.0}
Training on balanced dataset

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Dataset: balanced -> Results for seed 2048: {'eval_loss': 1.5275442600250244,
'eval_f1': 0.8292885638297872, 'eval_accuracy': 0.8354430379746836,
'eval_runtime': 0.2526, 'eval_samples_per_second': 625.6,
'eval_steps_per_second': 79.19, 'epoch': 5.0}

## Step 4 - Test Evaluation Here, models and tokenizers are being retrieved from the saved
directory and then evaluation on test set is being applied.

```

```

[ ]: def compute_metrics_test(output_info):
    predictions, labels = output_info

    f1 = f1_metric.compute(predictions=predictions, references=labels,
↪average="macro")
    acc = acc_metric.compute(predictions=predictions, references=labels)
    return {**f1, **acc}

# Path to saved models
results_dir = Path(transformer_results_path)

# Evaluation results
test_results = []

# os.environ["WANDB_DISABLED"] = "true"

```



```

eval_args = TrainingArguments(output_dir="temp_eval_output", report_to="none")

for seed in seeds:
    for dataset_name in datasets.keys():
        # Retrieving Model and Tokenizer
        model_dir = results_dir / f"{dataset_name}_seed_{seed}" / "model"
        tokenizer_dir = results_dir / f"{dataset_name}_seed_{seed}" / _
        ↪ "tokenizer"

        model = AutoModelForSequenceClassification.from_pretrained(model_dir)
        tokenizer = AutoTokenizer.from_pretrained(tokenizer_dir)

        # Initialize a new Trainer for inference
        trainer = Trainer(model=model, tokenizer=tokenizer, args=eval_args) #_
        ↪ This is also deprecating

        # Predict on test data
        predictions = trainer.predict(test_data)

        # Extract metrics and predictions
        metrics = predictions.metrics
        pred_labels = predictions.predictions.argmax(axis=-1)
        true_labels = test_data["label"] # True labels from test data

        # Save results for this seed
        test_results.append(
            {
                "seed": seed,
                "dataset": dataset_name,
                "metrics": metrics,
                "predictions": pred_labels.tolist(),
                "true_labels": true_labels,
            }
        )

# Print all test evaluation results
res_avg = {"unbalanced": [[], []], "balanced": [[], []]}
for result in test_results:
    output = (np.array(result["predictions"]), np.array(result["true_labels"]))
    test_met = compute_metrics_test(output)
    res_avg[result["dataset"]][0].append(test_met["f1"])
    res_avg[result["dataset"]][1].append(test_met["accuracy"])
    print(f" Seed: {result['seed']}, Dataset: {result['dataset']}, Test Metrics:
    ↪ {test_met}")

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

```
Seed: 512, Dataset: unbalanced, Test Metrics: {'f1': 0.823482940798894,
'accuracy': 0.8251748251748252}
Seed: 512, Dataset: balanced, Test Metrics: {'f1': 0.8386558744174638,
'accuracy': 0.8391608391608392}
Seed: 1024, Dataset: unbalanced, Test Metrics: {'f1': 0.8373714398734178,
'accuracy': 0.8391608391608392}
Seed: 1024, Dataset: balanced, Test Metrics: {'f1': 0.8268914829226113,
'accuracy': 0.8286713286713286}
Seed: 2048, Dataset: unbalanced, Test Metrics: {'f1': 0.8376043055349824,
'accuracy': 0.8391608391608392}
Seed: 2048, Dataset: balanced, Test Metrics: {'f1': 0.8281987471037502,
'accuracy': 0.8286713286713286}
```

```
[ ]: for datasets, metrics in res_avg.items():
      print("Dataset: ", datasets)
      print(f"Average F1 Score for {datasets}: {np.mean(metrics[0])}")
      print(f"Average Accuracy for {datasets}: {np.mean(metrics[1])}\n")
```

Dataset: unbalanced

Average F1 Score for unbalanced: 0.832819562069098

Average Accuracy for unbalanced: 0.8344988344988344

Dataset: balanced

Average F1 Score for balanced: 0.8312487014812749

Average Accuracy for balanced: 0.8321678321678322

The above results are reported in the transformer table in the report file.

An amazing performance is being performed by the transformer. Transformer is getting the accuracy of **~0.83** on the test set

As we have 3 seeds for robustness, in the next cell, we have applied a majority voting for the predictions on each tweets and as the results we came up with two arrays of predictions one for the balance dataset and another for the unbalance dataset.

```
[ ]: predictions_in_each_seed = {"unbalanced": [], "balanced": []}
      for each_test_result in test_results:
          predictions_in_each_seed[each_test_result["dataset"]].
          ↪append(each_test_result["predictions"])

      def pred_voting_for_transformer(list_of_predictions):
```

```

count = Counter(list_of_predictions)
if count[0] > count[1]:
    return 0
elif count[1] > count[0]:
    return 1
else:
    return 0

average_predictions_in_each_seed = {"unbalanced": [], "balanced": []}
for each_dataset in predictions_in_each_seed.keys():
    transformer_majority_vote = []
    transformer_list_of_pred_for_that_idx = []
    for voting_for_index in
↳range(len(predictions_in_each_seed["unbalanced"][0])):
        transformer_list_of_pred_for_that_idx = [
            pred_of_each_seed[voting_for_index] for pred_of_each_seed in
↳predictions_in_each_seed[each_dataset]
        ]
        transformer_majority_vote.
↳append(pred_voting_for_transformer(transformer_list_of_pred_for_that_idx))
    average_predictions_in_each_seed[each_dataset] = transformer_majority_vote

```

Results Analysis

BLSTM Curve Analysis Following cell visualizes the accuracy curve of both training and validation of the BLSTM.

```

[ ]: for model_name, metrics in average_acc_model_base.items():
    print(" ".join(model_name.split("_")), "----- # ----- # \n\n")
    print("Training Accuracy: ", metrics["acc"])
    print("Validation: ", metrics["val_acc"])

```

```

[ ]: for model_name, metrics in average_acc_model_base.items():
    print(
        "\n\n# ----- # -----",
        " ".join(model_name.split("_")),
        "----- # ----- # \n\n",
    )
    # Seperating validation and training accuracy
    acc = metrics["acc"]
    val_acc = metrics["val_acc"]

    # Seperating validation and training loss
    loss = metrics["loss"]
    val_loss = metrics["val_loss"]

    # Plotting

```

```

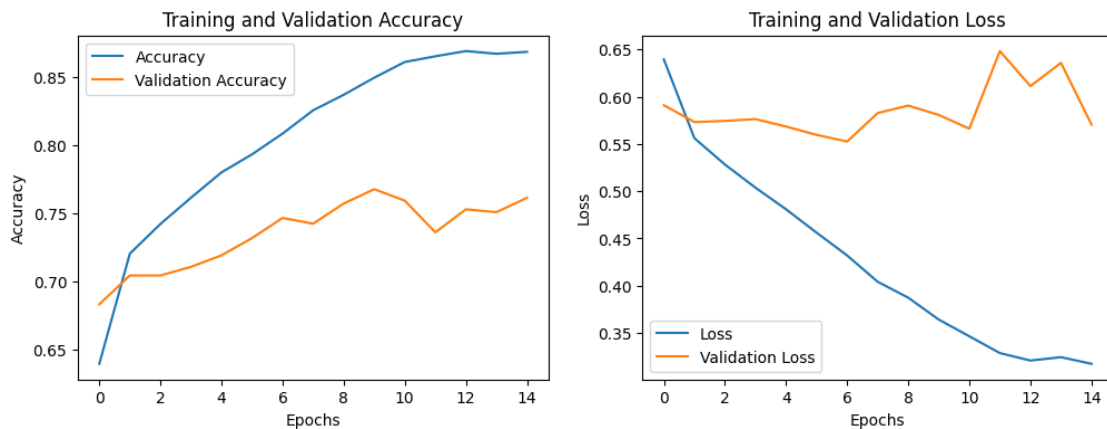
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(acc)
plt.plot(val_acc)
plt.title("Training and Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Accuracy", "Validation Accuracy"])

plt.subplot(1, 2, 2)
plt.plot(loss)
plt.plot(val_loss)
plt.title("Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Loss", "Validation Loss"])

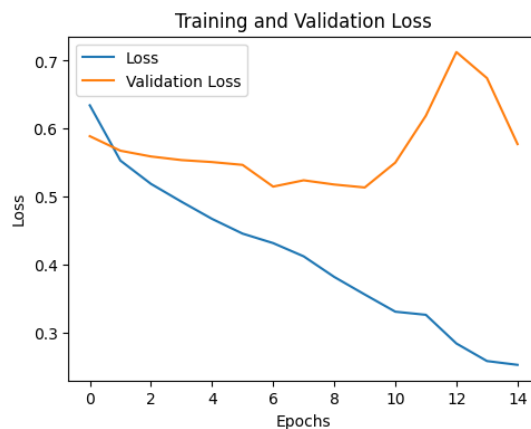
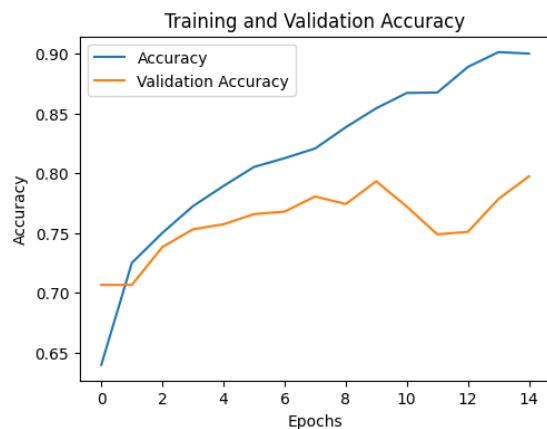
plt.show()

```

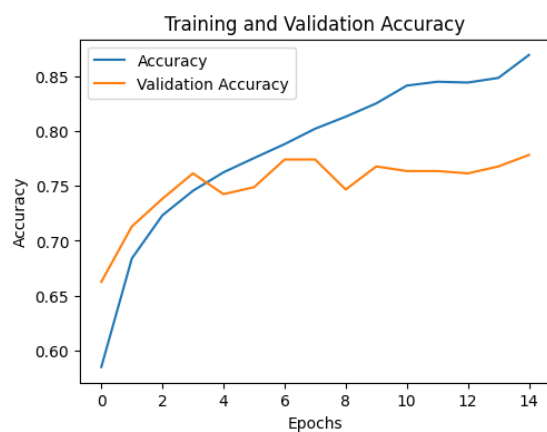
----- # ----- Model Base U ----- # -----



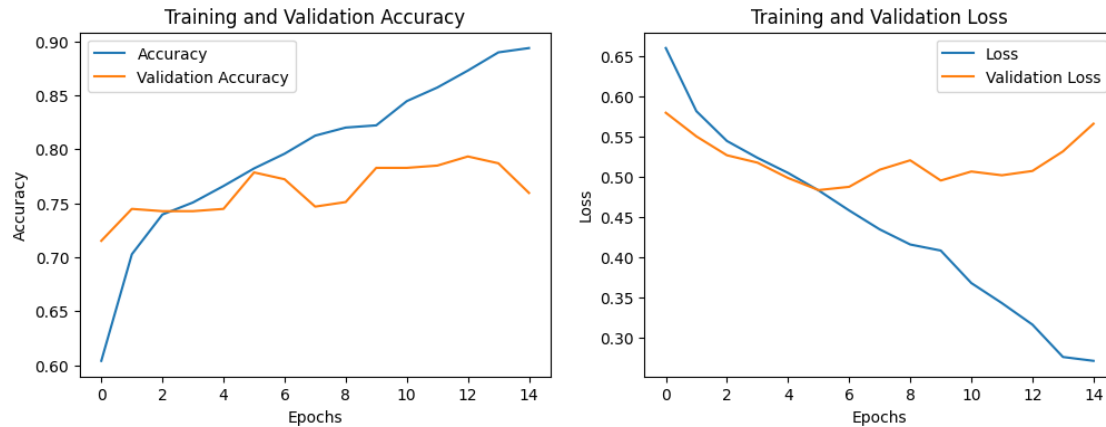
----- # ----- Model Base UA ----- # -----



----- # ----- Model Base B ----- # -----



----- # ----- Model Base BA ----- # -----



The losses of both the training and validation data on all of the models have almost a same structure. Training loss for all the four different models is decreasing epoch after epoch. However, the validation loss, almost in all the four models, starts increasing after epoch ~5.

- This suggests the models are overfitting to the training data and are losing the generalization abilities.

The same is also true for the accuracy section. While the accuracy on the training is getting better and better after each epoch, the accuracy of validation data is not experiencing a better results and even in some cases it gets worse after epoch ~8.

Vocabulary and Tokenization Following cell shows the total number of tokens that are unknown to the model.

```
[ ]: print("OOV in unbalanced training set:", check_OOV(train_sequences,
    ↪index_to_word))
print("OOV in unbalanced validation set:", check_OOV(val_sequences,
    ↪index_to_word))
print("OOV in unbalanced test set:", check_OOV(test_sequences, index_to_word))
print("-----")
print("OOV in balanced training set:", check_OOV(train_balanced_sequences,
    ↪index_to_word))
print("OOV in balanced validation set:", check_OOV(val_balanced_sequences,
    ↪index_to_word))
print("OOV in balanced test set:", check_OOV(test_balanced_sequences,
    ↪index_to_word))
```

```
OOV in unbalanced training set: 0
OOV in unbalanced validation set: 295
OOV in unbalanced test set: 456
-----
OOV in balanced training set: 0
OOV in balanced validation set: 315
OOV in balanced test set: 473
```

Why there is a difference between the number of OOV of balanced and unbalanced data? This is because, unbalanced data is the whole dataset, a larger dataset, which means there are more tokens inside it. As the results, the model has a larger vocabulary list. However, in the balanced dataset we have narrowed down the larger class, meaning that there were tweets that we didn't see them and we haven't add them to the vocabulary. Leading to a smaller vocabulary list.

In this project we have tokenized the data based on the word (Word Tokenization technique) where we choose all the preprocessed, lemmatized word types as a token. However, in modern Language Models, tokenization are being cared more carefully. - Changing the strategy for the tokenization also helps in encountering less OOV which helps the model to understand the data better. Having said that, we can say that one of the reason that transformer is doing better in this task, here in this project, is the different tokenization it uses.

Confusion Matrix Next, we'll define the metric functions and demonstrate the confusion matrix.

```
[ ]: acc_metric = evaluate.load("accuracy")
      f1_metric = evaluate.load("f1")

def compute_metrics(responses, y_true):
    """
    This function takes predicted and ground-truth labels and compute metrics.
    In particular, this function compute accuracy and fail-ratio metrics.
    This function internally invokes `process_response` to compute metrics.

    Inputs:
        responses: generated LLM responses
        y_true: ground-truth binary labels

    Outputs:
        dictionary containing desired metrics
    """
    f1 = f1_metric.compute(predictions=responses, references=y_true,
        ↪average="macro")
    acc = acc_metric.compute(predictions=responses, references=y_true)
    return {**f1, **acc}
```

```
[ ]: ground_truth = np.array(test_df["hard_label_task1"])
      preds_list = {}

for lstm_model_name in LSTM_majority_voting_predictions.keys():
    prediction = LSTM_majority_voting_predictions[lstm_model_name]
    preds_list[lstm_model_name] = prediction

for dataset_name in average_predictions_in_each_seed.keys():
    preds_list[f"Transformer on {dataset_name} dataset"] =
    ↪average_predictions_in_each_seed[dataset_name]
```

```

fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(10, 3 * 4))
axes = axes.ravel()

for idx, model_name in enumerate(preds_list.keys()):
    predictions = preds_list[model_name]

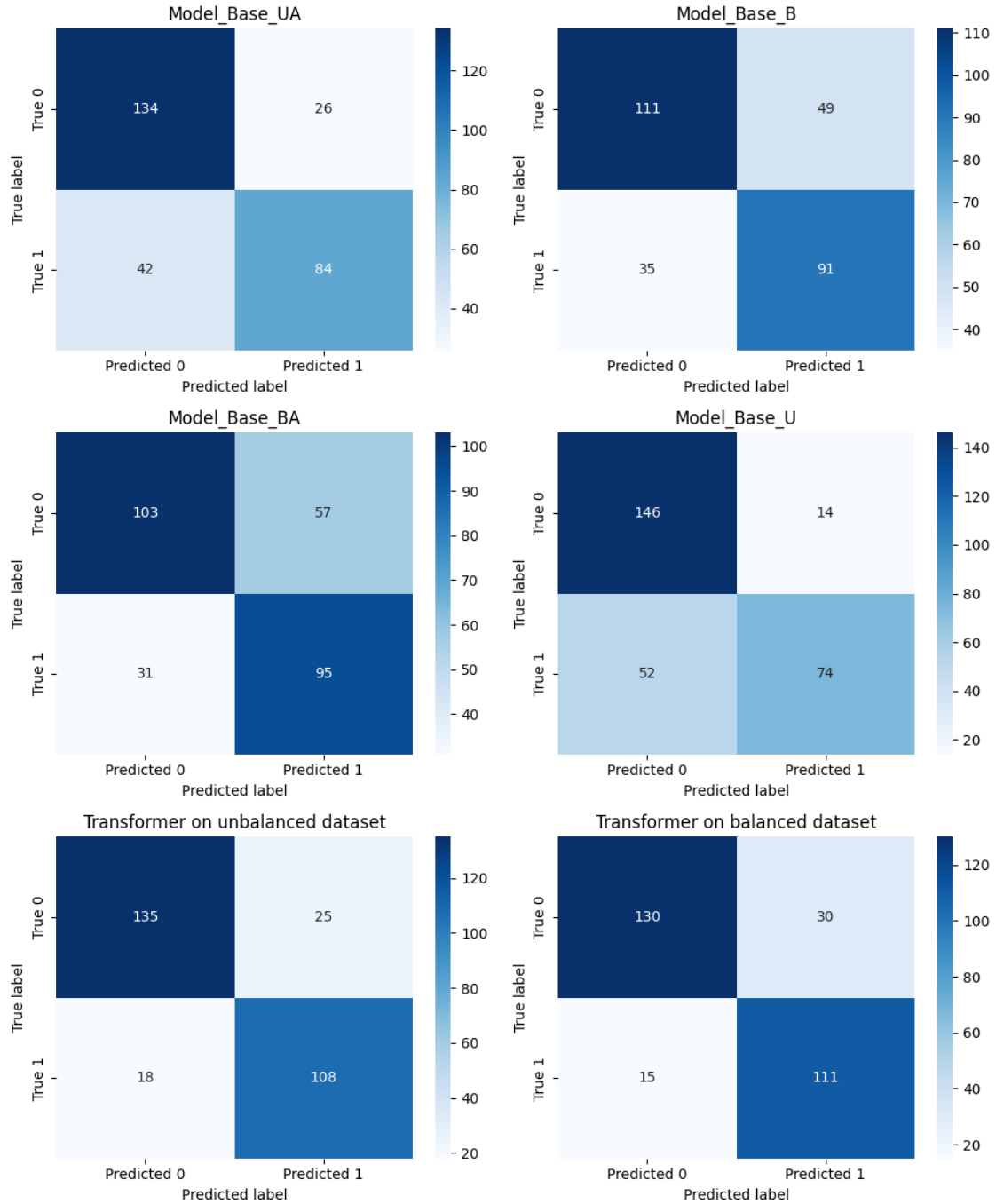
    cm = confusion_matrix(ground_truth, predictions)

    sns.heatmap(
        cm,
        annot=True,
        cmap="Blues",
        fmt="d",
        xticklabels=["Predicted 0", "Predicted 1"],
        yticklabels=["True 0", "True 1"],
        ax=axes[idx],
    )

    axes[idx].set_title(f"{model_name}")
    axes[idx].set_ylabel("True label")
    axes[idx].set_xlabel("Predicted label")

plt.tight_layout()
plt.show()

```

By studying these diagrams we can get a good understanding on the difference of the datasets that we have worked on.

BLSTM Diagram Analysis Here's the corresponding meaning of each name of the diagrams:

- Model Base U : The Base BLSTM; trained on **unbalance** data - Model Base UA : The BLSTM with Additional LSTM layer; trained on **unbalance** data - Model Base B : The Base BLSTM;

trained on **balance** data - Model Base BA : The BLSTM with Additional LSTM layer; trained on **balance** data

Diagrams for the unbalanced datasets are showing that the models is performing significantly better on predicting the tweets with the class 0, none sexist tweets. As expected, this is due the more data that we have for the class 0. Additionally, between these two models trained on this dataset, the model with additional LSTM layer is performing poorer than the base model. This is due to the overfitting that happens with more layers. Because the amount of data for such architecture is very less.

However, unexpectedly, the model trained on the balanced data are also performing better on classifying the non-sexist tweets. This may be due to the randomness, since during development, we've seen so many changes in these numbers. However, this attitude of cannot strongly be assigned to the randomness. One other reason that may causes this better performing on the class 0 may be because of the validation set. We also had the unbalanced data on the validation set. Although the model is not learning directly from the validation set, validation set is the set that helps model to update its weights and a bias in the validation set indeed can infect the training.

Transformer Diagram Analysis At the third row we have the confusion matrix of the transformer. Which is a showing that transformers are indeed more powerful than the BLSTM.

The difference between the transformer trained on the balanced data (the one on the right) and the transformer trained on the unbalanced data (the diagram on the left) with the results of the BLSTM architecuters can prove that, no matter how strong the architecture is, as long as the quality of the data is not good, or we don't have enough data for training, the model cannot perform better after some points.

Here, confusion matrices are showing us that transformer makes mistakes less than the BLSTM, but it is not performing significantly better. As mentioned, this may have different reasons, and one of them is the amount of data and the quality of data. The other one is the amount of training. A model overfitted to the training data cannot perform well on unseen data.

Other Notes Other notes:

- One of the things that can improve the performance is to augmente the data. Data augmentation can be used to increase the amount of data so our model doesn't overfit the data easily.
- Data Balancing is an important step that should be considered as well. Although we have experienced in some cases that the accuracy of the models on the unbalanced dataset was better, however, we must not rely on it. Because having an unbalanced data leads to a bias model, which maybe good even when you test it on test set, but cannot rely on it since the model is able to predict one class better than the other. Hence, we cannot say that the performance of the model is good because of the good predictino it has on that class. But on the other side, it is performing poor.
- One other things that could be exploit from is to use the grid search strategy to find the better parameters for the model. Although, still, when we don't have enough data, we cannot change the performance significantly better.
- One other things that can be done is to to instead of down sampling the majority class to balance the data, we could upsample the minority class the to have the same amount of examples as the mjority class. This increases the amount of data a bit.

- This upsampling is also can be complex as the characteristic of text is complex. However, finding the similarities between words and even sentences, could help us increase the number of examples we have for the minority class.